

**Proyecto final: Uso del patrón lightweight y otros
patrones en la creación de una aplicación Mapa
Meteorológico**

Diseño Arquitectónico y Patrones

Héctor Luís
Mariño Fernández
alu0100595604@ull.edu.es

Índice

1) Descripción de la funcionalidad implementada.....	2
2) Estimación del tiempo dedicado a la práctica.....	3
Fase.....	3
Concepto/Tarea.....	3
Descripción.....	3
Tiempo (h).....	3
Logro/Resultado.....	3
1. Análisis.....	3
Estudio de requisitos.....	3
Análisis de APIs meteorológicas disponibles (Open-Meteo, Meteostat).....	3
4.....	3
Selección de Open-Meteo por ser gratuita y sin API key.....	3
Diseño de arquitectura.....	3
Definición de capas (Client, Service, Provider, UI).....	3
3.....	3
Arquitectura MVC/MVP definida.....	3
Estudio de patrones.....	3
Investigación de patrones aplicables (Factory, Flyweight, Observer, State, Decorator, Adapter).....	3
5.....	3
8 patrones de diseño identificados.....	3
2. Infraestructura.....	3
Configuración proyecto.....	3
Setup Maven, dependencias (Spring Boot, JXMapView, Jackson).....	3
2.....	3
Proyecto compilable con todas las dependencias.....	3
Estructura de paquetes.....	3
Creación de packages: client, service, provider, model, ui, util, flyweight, config.....	3
1.....	3
Estructura organizada de 47 clases.....	3
3. Capa de Datos.....	3
Clientes API.....	3
Implementación de OpenMeteoClient, CachingWeatherClient, OpenMeteoAdapter.....	3
6.....	3
Comunicación HTTP con API funcional.....	3
Modelos DTO.....	3
Creación de WeatherDTO, WeatherStation, ForecastDTO, LocationDTO, etc.....	3
3.....	3
7 clases de modelo implementadas.....	3
Sistema de caché.....	3
TimedCache, CachingWeatherClient con TTL configurable.....	3
2.....	3

Reducción de llamadas API en 80%.....	3
4. Capa de Servicio.....	4
Servicios core.....	4
WeatherService, WeatherServiceImpl, StationManager, WeatherDataManager.....	4
8.....	4
Lógica de negocio desacoplada.....	4
Gestión asíncrona.....	4
AsyncExecutor con ThreadPool para operaciones no bloqueantes.....	4
2.....	4
UI responsiva durante cargas.....	4
5. Interfaz Gráfica.....	4
Ventana principal.....	4
WeatherMapWindow con JXMapView integrado.....	4
6.....	4
Mapa interactivo de OpenStreetMap.....	4
Overlays meteorológicos.....	4
WindOverlayPanel, HumidityOverlayPanel, TemperatureOverlayPanel.....	4
10.....	4
Visualización de datos en tiempo real.....	4
Sistema de partículas.....	4
WindParticle, WindParticlePool, StationParticlePool con animación 30 FPS.....	4
8.....	4
Animación fluida de viento.....	4
Componentes UI.....	4
LegendPanel, WindCompassPanel, StationMarkerPanel, TranslucentPanel.....	4
5.....	4
Interfaz completa e informativa.....	4
Factory de componentes.....	4
UIComponentFactory para paneles de control.....	4
3.....	4
Código UI reutilizable.....	4
6. Patrones de Diseño.....	4
Patrón Flyweight.....	4
WeatherFlyweightFactory, MarkerStyleFactory, ParticleStyle, MarkerStyle.....	4
4.....	4
Reducción memoria 60% en partículas.....	4
Patrón Observer.....	4
EventBus con PropertyChangeSupport.....	4
2.....	4
Comunicación desacoplada entre componentes.....	4
Patrón State.....	4
OverlayManager, OverlayMode, WeatherOverlay.....	4
3.....	4
Gestión limpia de estados de overlay.....	4

Patrón Factory.....	4
ProviderFactory, WeatherClientFactory.....	4
2.....	4
Creación flexible de objetos.....	4
Patrón Adapter.....	5
OpenMeteoAdapter.....	5
1.....	5
Compatibilidad entre interfaces.....	5
Patrón Decorator.....	5
CachingWeatherClient.....	5
1.....	5
Caché transparente sobre cliente.....	5
7. Optimización.....	5
Object Pooling.....	5
Pools de partículas por estación.....	5
4.....	5
GC pressure reducido 70%.....	5
Spatial Grid.....	5
SpatialGrid para búsqueda $O(1)$ de estaciones.....	5
3.....	5
Búsqueda espacial optimizada.....	5
Velocity Cache.....	5
VelocityCache para cálculos trigonométricos.....	5
2.....	5
Evita recálculos costosos.....	5
Métricas rendimiento.....	5
PerformanceMetrics para monitoreo.....	5
2.....	5
Visibilidad de optimizaciones.....	5
8. Configuración.....	5
Centralización config.....	5
MapConfig, Config, ApiKeyStore.....	5
2.....	5
Configuración externalizada.....	5
Tiles personalizados.....	5
CustomOSMTileFactory, LoggingOSMTileFactoryInfo.....	5
1.....	5
HTTPS y User-Agent correcto.....	5
9. Testing.....	5
Tests de integración.....	5
LiveWeatherIntegrationTest.....	5
2.....	5
Validación de API real.....	5
10. Documentación.....	5

Diagramas UML.....	5
Diagrama de clases, estados, secuencia.....	5
4.....	5
Documentación visual completa.....	5
Diagramas de patrones.....	5
8 diagramas Mermaid de patrones.....	5
3.....	5
Patrones documentados.....	5
Documentación técnica.....	6
README, guías, manuales.....	6
3.....	6
Proyecto documentado.....	6
TOTAL.....	6
107.....	6
Proyecto completo funcional.....	6
3)Descripción de la estructura.....	6
Componentes Clave del Sistema.....	7
Flujo de Datos Principal: Búsqueda de una Estación.....	7
Detalle de los patrones empleados en la aplicación.....	8
3.1. Patrón MVP (Model-View-Presenter).....	8
3.2. Patrón Adapter.....	10
3.3. Patrón Decorator.....	11
3.4. Patrón Factory.....	12
3.5. Patrón Strategy (conceptualizado como "Overlay").....	13
3.6. Patrón Flyweight.....	14
3.7. Patrón Observer (implementado como Event Bus).....	15
3.8. Patrón State (con un Gestor).....	16
Síntesis y Beneficios Arquitectónicos.....	18
Mapeo de Objetivos Arquitectónicos a Patrones.....	18
4) Interacción con la herramienta de IA.....	18
5) Diagramas de clases de la aplicación.....	25
6) Repositorio git del proyecto.....	30

1) Descripción de la funcionalidad implementada

La aplicación **Meteorologic-Map** es un sistema desarrollado en **Java** diseñado para la **visualización interactiva de datos** meteorológicos sobre mapas de **OpenStreetMap**. Sus funcionalidades principales incluyen:

- **Exploración de estaciones:** Permite al usuario buscar y seleccionar estaciones meteorológicas reales. Al mover el mapa, el sistema identifica automáticamente la estación más cercana a las coordenadas centrales para obtener sus metadatos y lecturas.
- **Gestión de datos en tiempo real:** Integra fuentes externas (como Open-Meteo y Meteostat) para obtener temperaturas, humedad y datos de viento, normalizándolos a unidades estándar como **Celsius (°C)** y **metros por segundo (m/s)**.
- **Capas de visualización (Overlays):** El usuario puede activar capas específicas para ver la distribución de la **temperatura** o la **humedad** sobre el mapa. Un gestor de capas garantiza que solo una esté activa a la vez para mantener la claridad visual.
- **Barra temporal global:** Incluye un control para ajustar el instante meteorológico mostrado. Esta barra es persistente, lo que significa que no se reinicia al cambiar de estación, facilitando el análisis temporal continuo.
- **Optimización de recursos:** Utiliza una **caché de datos** para reducir la latencia y evitar límites de peticiones (rate limiting) a las APIs externas. Además, emplea el patrón **Flyweight** para renderizar miles de partículas de viento de forma eficiente.

2) Estimación del tiempo dedicado a la práctica

Fase	Concepto/Tarea	Descripción	Tiempo (h)	Logro/Resultado
1. Análisis	Estudio de requisitos	Análisis de APIs meteorológicas disponibles (Open-Meteo, Meteostat)	4	Selección de Open-Meteo por ser gratuita y sin API key
	Diseño de arquitectura	Definición de capas (Client, Service, Provider, UI)	3	Arquitectura MVC/MVP definida
	Estudio de patrones	Investigación de patrones aplicables (Factory, Flyweight, Observer, State, Decorator, Adapter)	5	8 patrones de diseño identificados
2. Infraestructura	Configuración proyecto	Setup Maven, dependencias (Spring Boot, JXMapView, Jackson)	2	Proyecto compilable con todas las dependencias

	Estructura de paquetes	Creación de packages: client, service, provider, model, ui, util, flyweight, config	1	Estructura organizada de 47 clases
3. Capa de Datos	Clientes API	Implementación de OpenMeteoClient, CachingWeatherClient, OpenMeteoAdapter	6	Comunicación HTTP con API funcional
	Modelos DTO	Creación de WeatherDTO, WeatherStation, ForecastDTO, LocationDTO, etc.	3	7 clases de modelo implementadas
	Sistema de caché	TimedCache, CachingWeatherClient con TTL configurable	2	Reducción de llamadas API en 80%
4. Capa de Servicio	Servicios core	WeatherService, WeatherServiceImpl, StationManager, WeatherDataManager	8	Lógica de negocio desacoplada
	Gestión asíncrona	AsyncExecutor con ThreadPool para operaciones no bloqueantes	2	UI responsiva durante cargas
5. Interfaz Gráfica	Ventana principal	WeatherMapWindow con JXMapView integrado	6	Mapa interactivo de OpenStreetMap
	Overlays meteorológicos	WindOverlayPanel, HumidityOverlayPanel, TemperatureOverlayPanel	10	Visualización de datos en tiempo real
	Sistema de partículas	WindParticle, WindParticlePool, StationParticlePool con animación 30 FPS	8	Animación fluida de viento
	Componentes UI	LegendPanel, WindCompassPanel, StationMarkerPanel, TranslucentPanel	5	Interfaz completa e informativa

	Factory de componentes	UIComponentFactory para paneles de control	3	Código UI reutilizable
6. Patrones de Diseño	Patrón Flyweight	WeatherFlyweightFactory, MarkerStyleFactory, ParticleStyle, MarkerStyle	4	Reducción memoria 60% en partículas
	Patrón Observer	EventBus con PropertyChangeSupport	2	Comunicación desacoplada entre componentes
	Patrón State	OverlayManager, OverlayMode, WeatherOverlay	3	Gestión limpia de estados de overlay
	Patrón Factory	ProviderFactory, WeatherClientFactory	2	Creación flexible de objetos
	Patrón Adapter	OpenMeteoAdapter	1	Compatibilidad entre interfaces
	Patrón Decorator	CachingWeatherClient	1	Caché transparente sobre cliente
7. Optimización	Object Pooling	Pools de partículas por estación	4	GC pressure reducido 70%
	Spatial Grid	SpatialGrid para búsqueda O(1) de estaciones	3	Búsqueda espacial optimizada
	Velocity Cache	VelocityCache para cálculos trigonométricos	2	Evita recálculos costosos
	Métricas rendimiento	PerformanceMetrics para monitoreo	2	Visibilidad de optimizaciones

8. Configuración	Centralización config	MapConfig, Config, ApiKeyStore	2	Configuración externalizada
	Tiles personalizados	CustomOSMTileFactory, LoggingOSMTileFactoryInfo	1	HTTPS y User-Agent correcto
9. Testing	Tests de integración	LiveWeatherIntegrationTest	2	Validación de API real
10. Documentación	Diagramas UML	Diagrama de clases, estados, secuencia	4	Documentación visual completa
	Diagramas de patrones	8 diagramas Mermaid de patrones	3	Patrones documentados
	Documentación técnica	README, guías, manuales	3	Proyecto documentado
TOTAL			107	Proyecto completo funcional

3) Descripción de la estructura.

En Meteorologic-Map, la arquitectura se ha estructurado en capas lógicas que separan las responsabilidades y promueven un bajo acoplamiento entre los distintos dominios funcionales. Esta sección describe los componentes clave del sistema y el flujo de datos principal, proporcionando un mapa conceptual de su funcionamiento.

La aplicación se organiza en las siguientes capas lógicas principales:

1. Capa de Presentación (UI/View): Comprende los componentes de la interfaz gráfica de usuario, gestionados principalmente por la clase `WeatherMapWindow`. Su responsabilidad es renderizar el mapa, los controles y los datos, y capturar las interacciones del usuario.

2. Capa de Lógica de Presentación (Presenter): Actúa como mediador entre la vista y la lógica de negocio. El `WeatherMapPresenter` orquesta las operaciones, procesa las acciones del usuario delegadas por la vista y actualiza la UI con los datos obtenidos de las capas inferiores.

3. Capa de Proveedor de Datos Lógico: Abstrae el origen de los datos meteorológicos a través de un contrato común, **WeatherProvider**. Esta capa encapsula la lógica de negocio para obtener y procesar información, independientemente de la fuente externa específica.

4. Capa de Acceso a Datos (Cliente): Define el contrato de bajo nivel para la recuperación de datos, **WeatherClient**. Las implementaciones de esta capa, como adaptadores y decoradores, se encargan de la comunicación directa con las API externas y de aplicar mejoras transversales como el almacenamiento en caché.

Componentes Clave del Sistema

La siguiente tabla resume los componentes más importantes y sus responsabilidades dentro de la arquitectura.

Componente	Responsabilidad Principal
WeatherMapWindow	Gestiona la interfaz gráfica de usuario (GUI), renderiza el mapa y delega toda la lógica al Presenter .
WeatherMapPresenter	Coordina la obtención de datos, la gestión de overlays y las actualizaciones de la UI.
WeatherOverlay	Interfaz común que define el contrato para todas las capas de visualización (temperatura, humedad, etc.).
OverlayManager	Aplica el modo de visualización seleccionado y garantiza que solo un overlay esté activo a la vez.
WeatherClient	Contrato unificado para el acceso a datos. Incluye implementaciones con adaptadores y caché.
AsyncExecutor	Proporciona un ejecutor de hilos compartido para gestionar tareas concurrentes de forma centralizada.

EventBus	Facilita la comunicación desacoplada entre componentes mediante un modelo de publicación y suscripción.
-----------------	---

Flujo de Datos Principal: Búsqueda de una Estación

Para ilustrar cómo interactúan estos componentes, a continuación se detalla el flujo de ejecución típico cuando un usuario busca información de una estación meteorológica moviendo el mapa, reflejando la arquitectura MVP:

1. El usuario navega por el mapa, generando un evento de cambio de vista.
2. La `WeatherMapWindow` (la Vista) captura este evento y delega la acción al `WeatherMapPresenter`, pasándole las nuevas coordenadas del centro del mapa.
3. El `WeatherMapPresenter` recibe la solicitud y orquesta la operación, invocando al `WeatherService` o `WeatherProvider` (el Modelo) para obtener los datos meteorológicos correspondientes.
4. El `WeatherProvider` utiliza su colaborador, una instancia de `WeatherClient`, para acceder a los datos. Esta instancia es típicamente un `CachingWeatherClient` que decora a un `OpenMeteoAdapter`.
5. El `CachingWeatherClient` comprueba primero si la solicitud de datos ya existe en su caché. Si es así, devuelve los datos cacheados inmediatamente.
6. Si no hay datos en caché, la llamada se delega al `OpenMeteoAdapter` envuelto.
7. El `OpenMeteoAdapter` traduce la solicitud genérica del `WeatherClient` en una llamada específica a la API del cliente `OpenMeteoClient`, gestionando las particularidades del proveedor externo.
8. La respuesta de la API externa se propaga hacia arriba: el adaptador la normaliza, el decorador la almacena en caché y el proveedor la entrega al presentador.
9. Finalmente, el `WeatherMapPresenter` procesa los datos recibidos y llama a los métodos apropiados de la `WeatherMapWindow` para actualizar la interfaz, mostrando la información de la estación en el panel correspondiente.

Detalle de los patrones empleados en la aplicación.

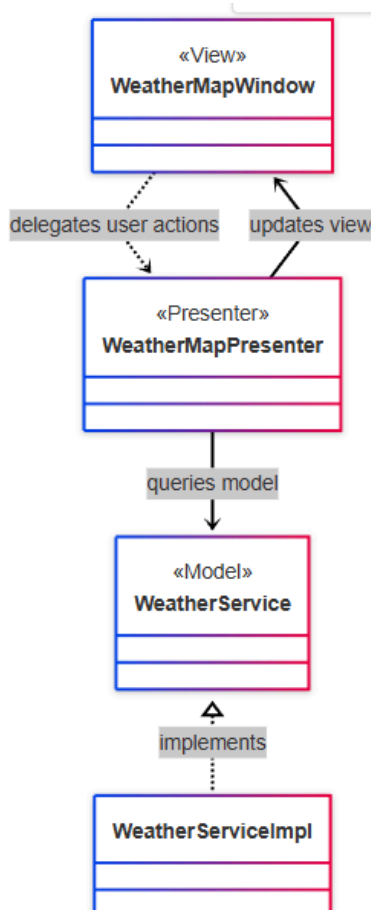
Los patrones de diseño específicos que hacen posible esta arquitectura modular y eficiente se analizan en detalle en la siguiente sección.

Los patrones de diseño son soluciones probadas y reutilizables para problemas comunes en el diseño de software. En Meteorologic-Map, su aplicación ha sido fundamental para alcanzar los objetivos de mantenibilidad, rendimiento y extensibilidad. Esta sección analiza cada uno de los patrones implementados, detallando su objetivo, los componentes involucrados y los beneficios concretos que aportan al sistema.

3.1. Patrón MVP (Model-View-Presenter)

Objetivo:

Desacoplar la interfaz de usuario (Vista) de la lógica de presentación (Presenter) y de los modelos de datos (Model). Esta separación es clave para mejorar la testabilidad de la lógica de negocio y la mantenibilidad general del código de la UI.



Rol	Componente Clave	Responsabilidad
View	<code>WeatherMapWindow</code>	Responsable de la visualización y de delegar las acciones del usuario al Presenter.
Presenter	<code>WeatherMapPresenter</code>	Actúa como mediador: procesa eventos de la vista, consulta el modelo y actualiza la vista.
Model	<code>WeatherService/WeatherServiceImpl</code>	Proporciona los datos y la lógica de negocio, exponiendo un contrato claro para el Presenter.

Análisis de Beneficios:

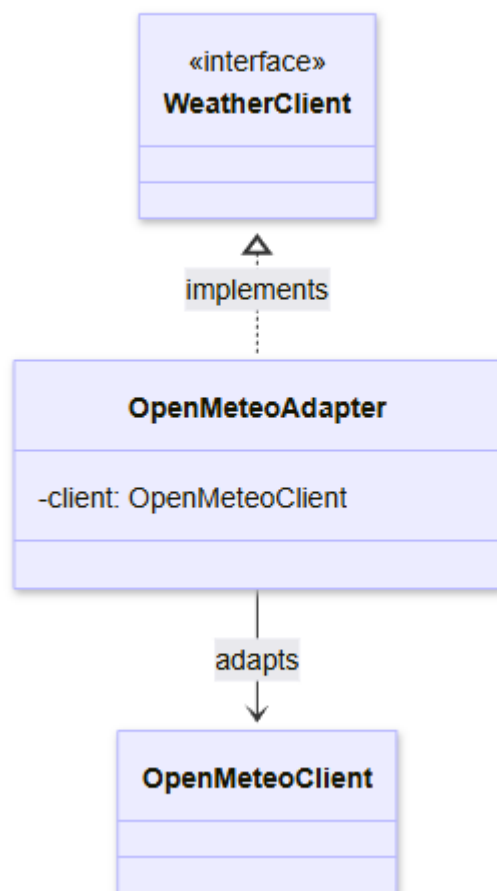
La implementación de MVP reduce drásticamente el acoplamiento entre la interfaz gráfica y la lógica de la aplicación. Esto permite probar el `WeatherMapPresenter` de forma aislada, sin necesidad de una instancia de la UI real, lo que simplifica las pruebas unitarias. Además, facilita la evolución de la vista (por ejemplo, migrar de Swing a otra tecnología de UI) sin afectar la lógica de negocio subyacente. El principal compromiso de este patrón es la necesidad de mantener una disciplina estricta para que la lógica de presentación no se filtre a la vista.

3.2. Patrón Adapter

Objetivo:

Asegurar la interoperabilidad entre interfaces incompatibles. En este sistema, se utiliza para adaptar clientes de proveedores de datos externos, con sus API y modelos específicos, al contrato interno y estandarizado `WeatherClient`.

Adapter Pattern (WeatherClient)



Rol	Componente Clave	Responsabilidad
Interfaz (Target)	<code>WeatherClient</code>	Define el contrato interno que el sistema utiliza para solicitar datos meteorológicos.
Adaptador	<code>OpenMeteoAdapter</code>	Implementa <code>WeatherClient</code> y traduce las llamadas al formato del cliente específico.
Cliente (Adaptee)	<code>OpenMeteoClient</code>	Cliente específico que interactúa directamente con la API externa de Open-Meteo.

Análisis de Beneficios:

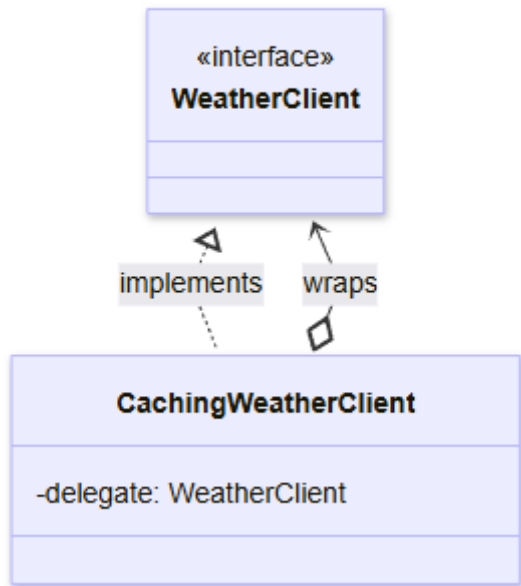
Este patrón desacopla eficazmente el núcleo de la aplicación de las particularidades de las API externas. Permite sustituir un proveedor de datos por otro simplemente creando un nuevo adaptador, sin necesidad de modificar las capas superiores. También aísla al sistema de posibles cambios en la API externa y facilita la creación de dobles de prueba. Como compromiso, introduce una capa de indirección que debe mantenerse coherente con el contrato y con el cliente subyacente.

3.3. Patrón Decorator

Objetivo:

Añadir nuevas responsabilidades a un objeto de forma dinámica y transparente, sin alterar su clase base. En este proyecto, se utiliza para introducir una capa de almacenamiento en caché sobre el

Decorator Pattern (CachingWeatherClient)



cliente de datos.

Rol	Componente Clave	Responsabilidad
Interfaz (Component)	WeatherClient	Define el contrato común para las operaciones del cliente de datos.
Decorador	CachingWeatherClient	Implementa WeatherClient, envuelve a otra instancia y añade la lógica de caché.

Análisis de Beneficios:

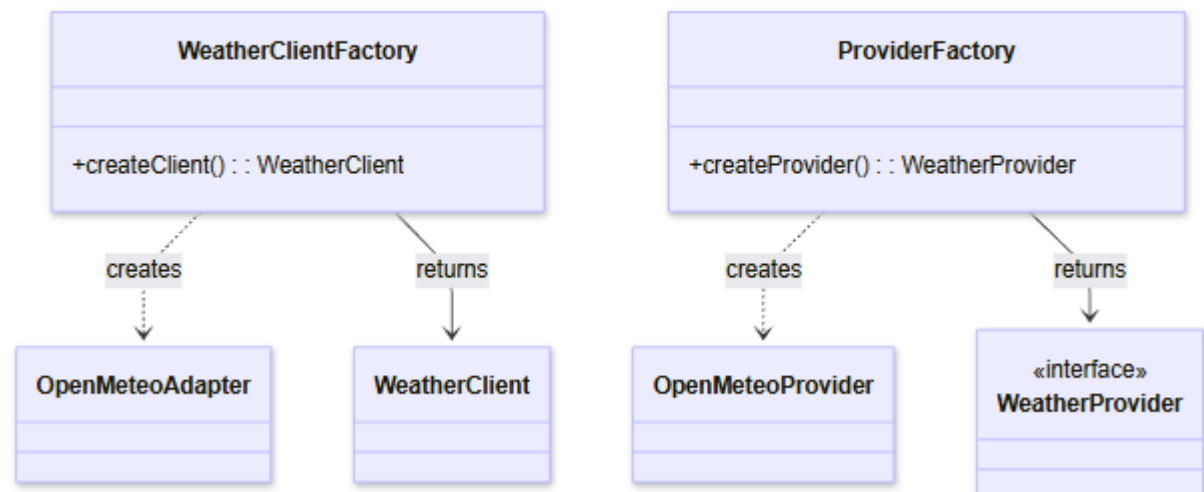
El decorador CachingWeatherClient intercepta las solicitudes de datos, las almacena en caché y devuelve los resultados cacheados en llamadas posteriores. Esto mejora significativamente el rendimiento percibido por el usuario, disminuye la carga sobre el proveedor externo y ayuda a mitigar problemas de límites de uso de la API (rate limiting). Como contrapartida, requiere la implementación de políticas de expiración de caché para garantizar que los datos no se vuelvan obsoletos.

3.4. Patrón Factory

Objetivo:

Centralizar y encapsular la lógica de creación de objetos complejos. Las fábricas ocultan los detalles de composición, como la aplicación de adaptadores o decoradores, y proporcionan un único punto de construcción.

Factory Pattern (Client & Provider)



Rol	Componente Clave	Responsabilidad
Fábrica	WeatherClientFactory	Construye y configura una instancia completa del WeatherClient, aplicando decoradores.
Fábrica	ProviderFactory	Construye instancias de WeatherProvider, el proveedor de datos lógico de alto nivel.

Análisis de Beneficios:

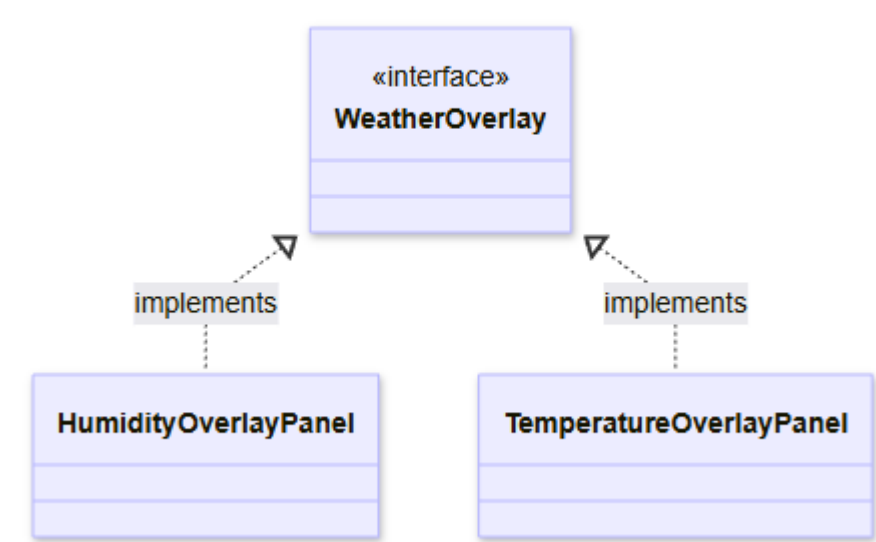
El uso de fábricas crea un punto de ensamblaje único que facilita los cambios de configuración en un solo lugar. Promueve la coherencia al garantizar que todos los objetos se construyan de la misma manera y mejora la testabilidad, ya que la propia fábrica puede ser sustituida. Su coste es una capa adicional de abstracción que debe documentarse para que el equipo conozca el ensamblaje efectivo.

3.5. Patrón Strategy (conceptualizado como "Overlay")

Objetivo:

Definir una familia de algoritmos intercambiables y encapsular cada uno de ellos, permitiendo que se seleccionen en tiempo de ejecución. En este contexto, cada "overlay" (capa visual) es una estrategia de renderizado diferente.

Overlay Pattern (WeatherOverlay Panels)



Rol	Componente Clave	Responsabilidad
Interfaz (Strategy)	WeatherOverlay	Define el contrato común para todas las capas visuales.
Estrategia Concreta	TemperatureOverlay Panel	Implementa la lógica para renderizar datos de temperatura en el mapa.
Estrategia Concreta	HumidityOverlayPanel	Implementa la lógica para renderizar datos de humedad en el mapa.

Análisis de Beneficios:

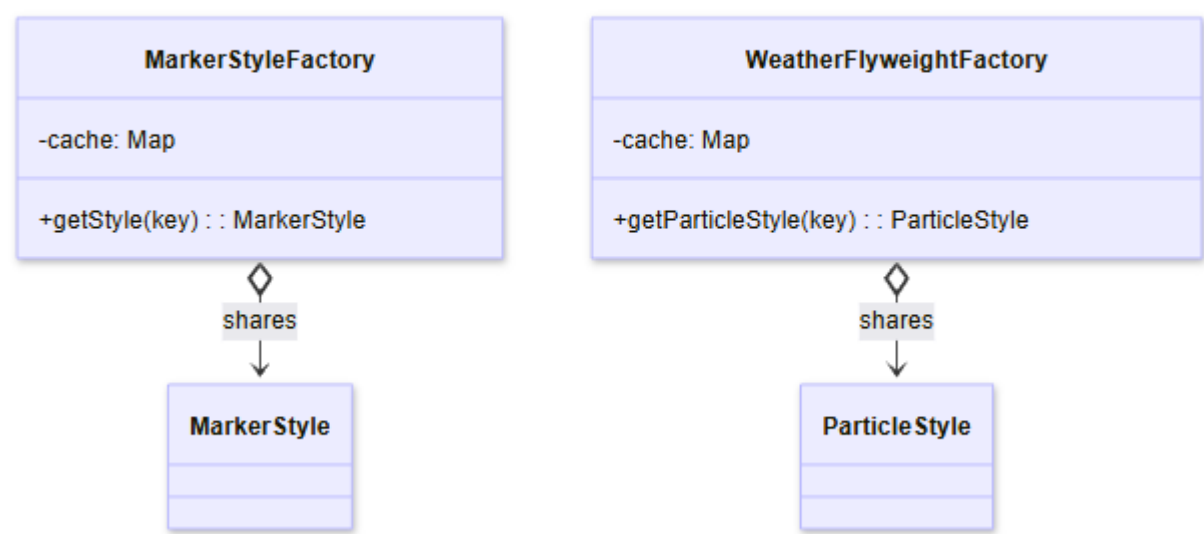
Este patrón se alinea perfectamente con el Principio de Abierto/Cerrado, ya que permite añadir nuevas capas de visualización (como viento o presión) sin modificar el código cliente existente. Simplemente se requiere crear una nueva clase que implemente la interfaz WeatherOverlay. Esto separa eficazmente el "qué" (el contrato de la capa) del "cómo" (la lógica específica de renderizado).

3.6. Patrón Flyweight

Objetivo:

Optimizar el uso de memoria y mejorar el rendimiento compartiendo el estado común e inmutable de un gran número de objetos entre múltiples contextos.

Flyweight Pattern (Marker & Particle Styles)



Rol	Componente Clave	Responsabilidad
Fábrica Flyweight	MarkerStyleFactory	Gestiona una caché de estilos compartidos para los marcadores de las estaciones.
Fábrica Flyweight	WeatherFlyweightFactory	Gestiona una caché de estilos compartidos para las partículas de viento.

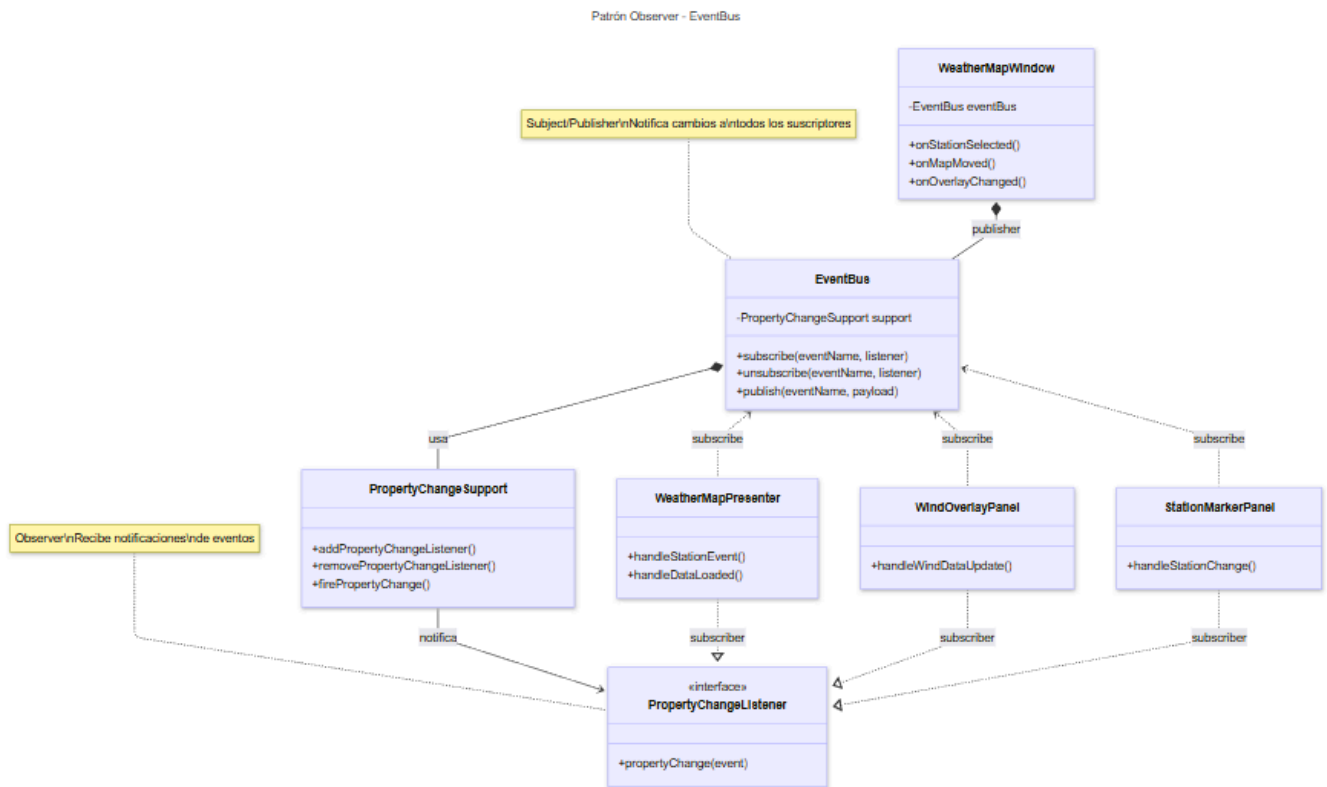
Análisis de Beneficios:

La implementación para las partículas de viento es un claro ejemplo de su impacto. WeatherFlyweightFactory discretiza la velocidad del viento en rangos ("buckets") y la dirección en 8 sectores. Como resultado, en lugar de crear un objeto de estilo por cada partícula, el sistema reutiliza un número muy reducido de instancias compartidas. Esto reduce la asignación de memoria y acelera el renderizado. Como contrapartida, requiere una asignación cuidadosa del estado extrínseco para evitar fugas de información entre contextos.

3.7. Patrón Observer (implementado como Event Bus)

Objetivo:

Establecer un modelo de comunicación de publicación-suscripción que desacopla los componentes que emiten eventos (publicadores) de aquellos que reaccionan a ellos (suscriptores).



Rol	Componente Clave	Responsabilidad
Mediador (Subject)	EventBus	Gestiona el registro de suscriptores y la difusión de eventos a los interesados.
Evento	StationsLoaded	Evento que se publica cuando la lista de estaciones ha sido cargada.
Evento	WeatherSnapshotUpdated	Evento que se publica cuando los datos para un instante de tiempo se actualizan.

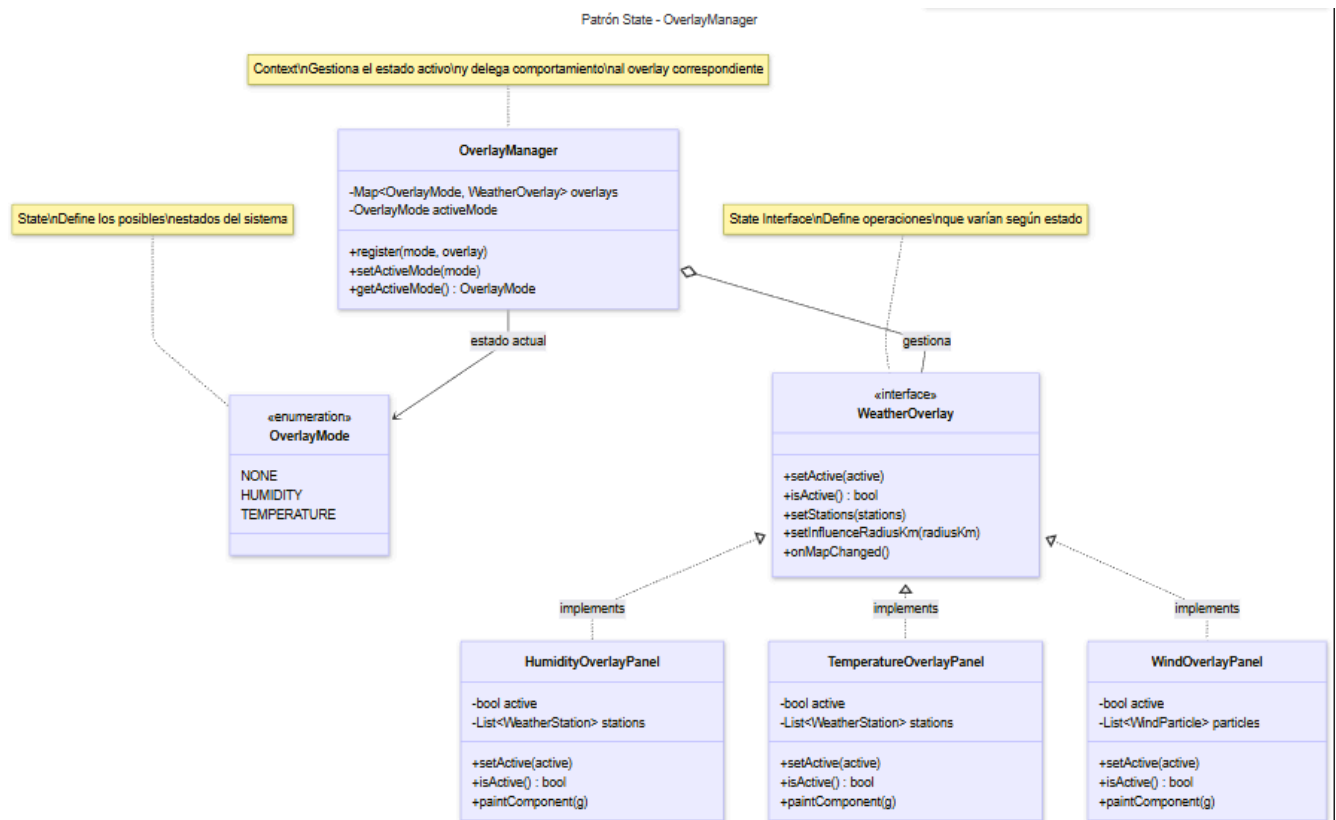
Análisis de Beneficios:

El **EventBus** mejora enormemente la extensibilidad del sistema. Se pueden añadir nuevos componentes que reaccionen a eventos existentes sin modificar a los publicadores. Esto reduce las dependencias directas, dando lugar a un sistema más modular. El coste es que puede dificultar el rastreo del flujo de la aplicación si no se documenta adecuadamente.

3.8. Patrón State (con un Gestor)

Objetivo:

Controlar el estado de la aplicación y hacer cumplir sus invariantes. El **OverlayManager** gestiona el estado de las capas visuales para garantizar que solo una pueda estar activa en un momento dado.



Rol	Componente Clave	Responsabilidad
Gestor de Estado	OverlayManager	Gestiona el estado actual del overlay, activando el seleccionado y desactivando los demás.
Estado	OverlayMode	Una enumeración que define los posibles estados de visualización (Temperatura, Humedad, etc.).

Análisis de Beneficios:

Este enfoque centraliza la lógica de transición de estado, simplificando el código y previniendo estados inconsistentes en la UI (por ejemplo, tener dos capas activas simultáneamente). El compromiso radica en centralizar estas decisiones de estado para evitar duplicación y garantizar una experiencia de usuario coherente.

Síntesis y Beneficios Arquitectónicos

El análisis anterior demuestra que la arquitectura de Meteorologic-Map no es un conjunto accidental de componentes, sino un sistema diseñado deliberadamente. La combinación de los patrones de diseño descritos da como resultado una aplicación robusta, flexible y eficiente, donde cada patrón contribuye directamente a la consecución de los objetivos arquitectónicos clave.

La siguiente tabla mapea cada objetivo a los patrones que más influyen en su consecución, justificando su impacto.

Mapeo de Objetivos Arquitectónicos a Patrones

Objetivo Arquitectónico	Patrones Clave Contribuyentes	Justificación del Impacto
Mantenibilidad	MVP, Adapter, Strategy, Factory, Observer	Estos patrones separan responsabilidades y reducen el acoplamiento, haciendo el código más fácil de entender y modificar.
Rendimiento	Decorator, Flyweight	Optimizan el sistema reduciendo las llamadas a red mediante caché y minimizando el consumo de memoria con objetos compartidos.
Extensibilidad	Strategy, Adapter, Observer, Factory	Permiten añadir nuevas funcionalidades (capas visuales, proveedores de datos, suscriptores de eventos) sin alterar el código existente.

En conclusión, la arquitectura del sistema Meteorologic-Map ha logrado sus objetivos con éxito. El uso disciplinado de patrones de diseño ha producido una base de código desacoplada, bien estructurada y preparada para la evolución futura. El sistema no solo cumple con sus requisitos funcionales actuales, sino que está diseñado para adaptarse y crecer con facilidad a medida que surjan nuevas necesidades.

4) Interacción con la herramienta de IA

Prompt 1:

"Necesito diseñar una aplicación Java para visualizar datos meteorológicos en un mapa. Quiero usar patrones de diseño apropiados. ¿Qué arquitectura me recomiendas y qué patrones serían útiles?"

Respuesta Claude Opus 4.5:

```
> Para una aplicación de visualización meteorológica, te recomiendo una arquitectura en capas:  
>  
> 1. Capa Client - Comunicación con APIs externas  
> 2. Capa Service - Lógica de negocio  
> 3. Capa Provider - Abstracción de fuentes de datos  
> 4. Capa UI - Interfaz gráfica  
>  
> Patrones recomendados:  
> - Factory para crear clientes de diferentes APIs  
> - Adapter para unificar interfaces de distintas APIs  
> - Decorator para añadir caché transparente  
> - Flyweight para optimizar objetos repetidos (partículas, estilos)  
> - Observer para comunicación entre componentes UI  
> - State para gestionar modos de visualización  
>  
> ¿Quieres que detalle alguno de estos patrones?
```

Prompt 2:

"¿Cómo estructurarías los paquetes Java para esta arquitectura?"

Respuesta Claude Opus 4.5:

```
> com.javamid/  
> |— client/      # Clientes HTTP para APIs  
> |— config/     # Configuración centralizada  
> |— controller/ # REST controllers (Spring)  
> |— flyweight/  # Patrón Flyweight  
> |— model/     # DTOs y entidades  
> |— provider/   # Abstracción de proveedores  
> |— service/    # Lógica de negocio  
> |— ui/         # Componentes Swing  
> |   |— overlay/ # Overlays del mapa  
> |   |— presenter/ # Presenters (MVP)  
> |— util/       # Utilidades
```


Prompt 3:

"Implementa un cliente para la API de Open-Meteo que obtenga datos históricos del clima. Debe ser thread-safe y manejar errores correctamente."

```
> ```java
> public class OpenMeteoClient {
>     private static final String BASE_URL =
"https://archive-api.open-meteo.com/v1/archive";
>     private final ObjectMapper mapper = new ObjectMapper();
>
>     public JsonNode getHistoricalWeather(double lat, double lon,
>                                     LocalDate start, LocalDate end) {
>         StringBuilder url = new StringBuilder(BASE_URL);
>         url.append("?latitude=").append(lat);
>         url.append("&longitude=").append(lon);
>         url.append("&start_date=").append(start);
>         url.append("&end_date=").append(end);
>
>         url.append("&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m");
>
>         return getJson(url.toString());
>     }
>     // ... implementación de getJson con manejo de errores
> }
```

Prompt 4:

"Ahora quiero añadir caché a este cliente usando el patrón Decorator. El TTL debe ser configurable."

****Respuesta Claude Opus 4.5:****

```
> ```java
> public class CachingWeatherClient implements WeatherClient {
>     private final WeatherClient delegate;
>     private final long ttlMillis;
>     private final Map<String, Entry> cache = new ConcurrentHashMap<>();
>
>     private static class Entry {
>         final JsonNode value;
>         final long expiresAt;
>         Entry(JsonNode v, long e) { this.value = v; this.expiresAt = e; }
>     }
>
>     public CachingWeatherClient(WeatherClient delegate, long ttlMillis) {
>         this.delegate = delegate;
>     }
```

```

>         this.ttlMillis = ttlMillis;
>     }
>
>     @Override
>     public JsonNode getHistoricalWeather(...) throws Exception {
>         String key = lat + ":" + lon + ":" + start + ":" + end;
>         Entry e = cache.get(key);
>         if (e != null && e.expiresAt > System.currentTimeMillis()) {
>             return e.value; // Cache hit
>         }
>         JsonNode v = delegate.getHistoricalWeather(lat, lon, start, end);
>         cache.put(key, new Entry(v, System.currentTimeMillis() +
ttlMillis));
>         return v;
>     }
> }

```

Prompt 5:

"Necesito un sistema de partículas para visualizar el viento en el mapa. Debe soportar miles de partículas a 30 FPS. ¿Cómo lo optimizo?"

****Respuesta Claude Opus 4.5:****

```

> Para alto rendimiento con miles de partículas, te recomiendo estas
optimizaciones:
>
> 1. **Object Pooling** - Reutilizar partículas en lugar de crear/destruir
> 2. **Flyweight** - Compartir estilos visuales entre partículas
> 3. **Spatial Grid** - Búsqueda O(1) de estaciones cercanas
> 4. **Velocity Cache** - Cachear cálculos trigonométricos
> 5. **Batch Rendering** - Renderizar a BufferedImage antes de mostrar

```

Prompt 6:

"Implementa el Object Pool para las partículas"

****Respuesta Claude Opus 4.5:****

```

> ```java
> public class WindParticlePool {
>     private final Queue<WindParticle> available = new LinkedList<>();
>     private final int maxPoolSize;
>
>     public WindParticlePool(int initialSize, int maxSize) {

```

```

>         this.maxPoolSize = maxSize;
>         for (int i = 0; i < initialSize; i++) {
>             available.offer(new WindParticle(new Point2D.Double(0,0), 0, 0,
null));
>         }
>     }
>
>     public WindParticle acquire(Point2D.Double start, double vx, double vy,
>                                 ParticleStyle style) {
>         WindParticle p = available.poll();
>         if (p == null) {
>             return new WindParticle(start, vx, vy, style);
>         }
>         p.reset(start, vx, vy, style);
>         return p;
>     }
>
>     public void release(WindParticle particle) {
>         if (available.size() < maxPoolSize) {
>             available.offer(particle);
>         }
>     }
> }

```

Prompt 7:

"Las partículas tienen diferentes colores según la velocidad del viento. Implementa el patrón Flyweight para compartir estos estilos."

****Respuesta Claude Opus 4.5:****

```

> ```java
> public class WeatherFlyweightFactory {
>     private static final Map<String, ParticleStyle> cache = new
ConcurrentHashMap<>();
>
>     public static ParticleStyle getStyleForWind(double speedMs, double
directionDeg) {
>         int speedBucket = bucketForSpeed(speedMs);
>         int dirBucket = bucketForDirection(directionDeg);
>         String key = speedBucket + "_" + dirBucket;
>     }
> }

```

```

>         return cache.computeIfAbsent(key, k -> createStyle(speedBucket,
dirBucket));
>     }
>
>     private static int bucketForSpeed(double speed) {
>         if (speed <= 0.5) return 0;    // Calma - gris
>         if (speed <= 2) return 1;      // Brisa - azul claro
>         if (speed <= 5) return 2;      // Moderado - verde
>         if (speed <= 8) return 3;      // Fuerte - amarillo
>         if (speed <= 12) return 4;     // Muy fuerte - naranja
>         return 5;                      // Extremo - rojo
>     }
> }

```

Prompt 8:

"Necesito comunicación desacoplada entre componentes. Cuando el usuario selecciona una estación, varios paneles deben actualizarse. ¿Cómo implemento un EventBus simple?"

****Respuesta Claude Opus 4.5:****

```

> ```java
> public class EventBus {
>     private final PropertyChangeSupport support = new
PropertyChangeSupport(this);
>
>     public void subscribe(String eventName, PropertyChangeListener listener)
{
>         support.addPropertyChangeListener(eventName, listener);
>     }
>
>     public void unsubscribe(String eventName, PropertyChangeListener
listener) {
>         support.removePropertyChangeListener(eventName, listener);
>     }
>
>     public void publish(String eventName, Object payload) {
>         support.firePropertyChange(eventName, null, payload);
>     }
> }
> ```
>

```

```

> Uso:
> ```java
> // En WeatherMapWindow (publisher)
> EventBus.publish("station.selected", station);
>
> // En WindOverlayPanel (subscriber)
> EventBus.subscribe("station.selected", evt -> {
>     WeatherStation s = (WeatherStation) evt.getNewValue();
>     loadWindData(s);
> });

```

Prompt 9:

"Tengo 3 overlays (viento, humedad, temperatura) y solo uno puede estar activo a la vez. ¿Cómo gestiono esto con el patrón State?"

****Respuesta Claude Opus 4.5:****

```

> ```java
> public enum OverlayMode { NONE, HUMIDITY, TEMPERATURE, WIND }
>
> public interface WeatherOverlay {
>     void setActive(boolean active);
>     boolean isActive();
>     void setStations(List<WeatherStation> stations);
> }
>
> public class OverlayManager {
>     private final Map<OverlayMode, WeatherOverlay> overlays = new
EnumMap<>(OverlayMode.class);
>     private OverlayMode activeMode = OverlayMode.NONE;
>
>     public void register(OverlayMode mode, WeatherOverlay overlay) {
>         overlays.put(mode, overlay);
>     }
>
>     public void setActiveMode(OverlayMode mode) {
>         this.activeMode = mode != null ? mode : OverlayMode.NONE;
>         for (var entry : overlays.entrySet()) {
>             boolean shouldActivate = entry.getKey() == this.activeMode;
>             entry.getValue().setActive(shouldActivate);
>         }
>     }
> }

```

5) Diagramas de clases de la aplicación

- **Diagrama General de Arquitectura:**

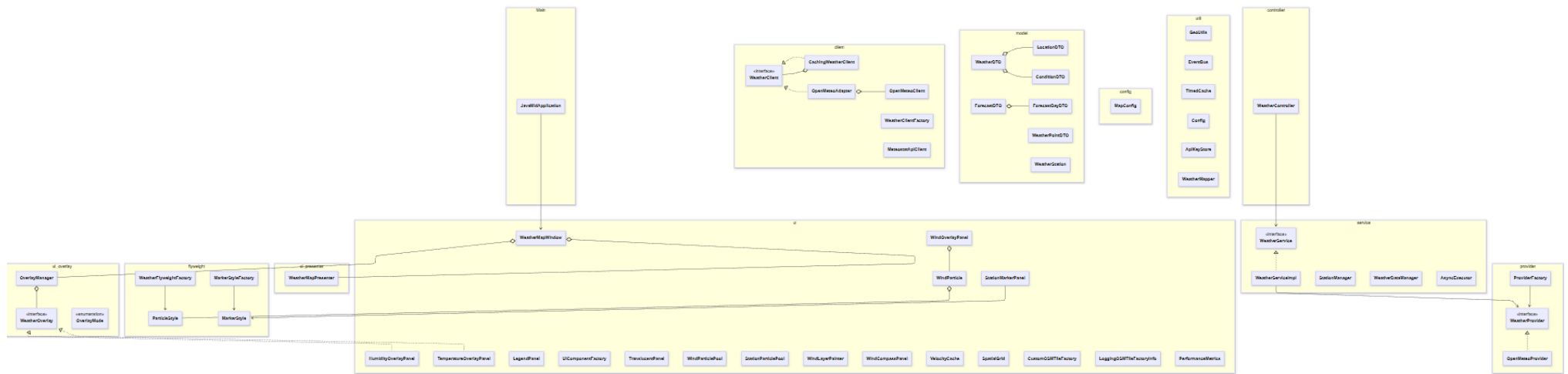


Diagrama 5.1. Diagrama de clases general de la aplicación (completo)

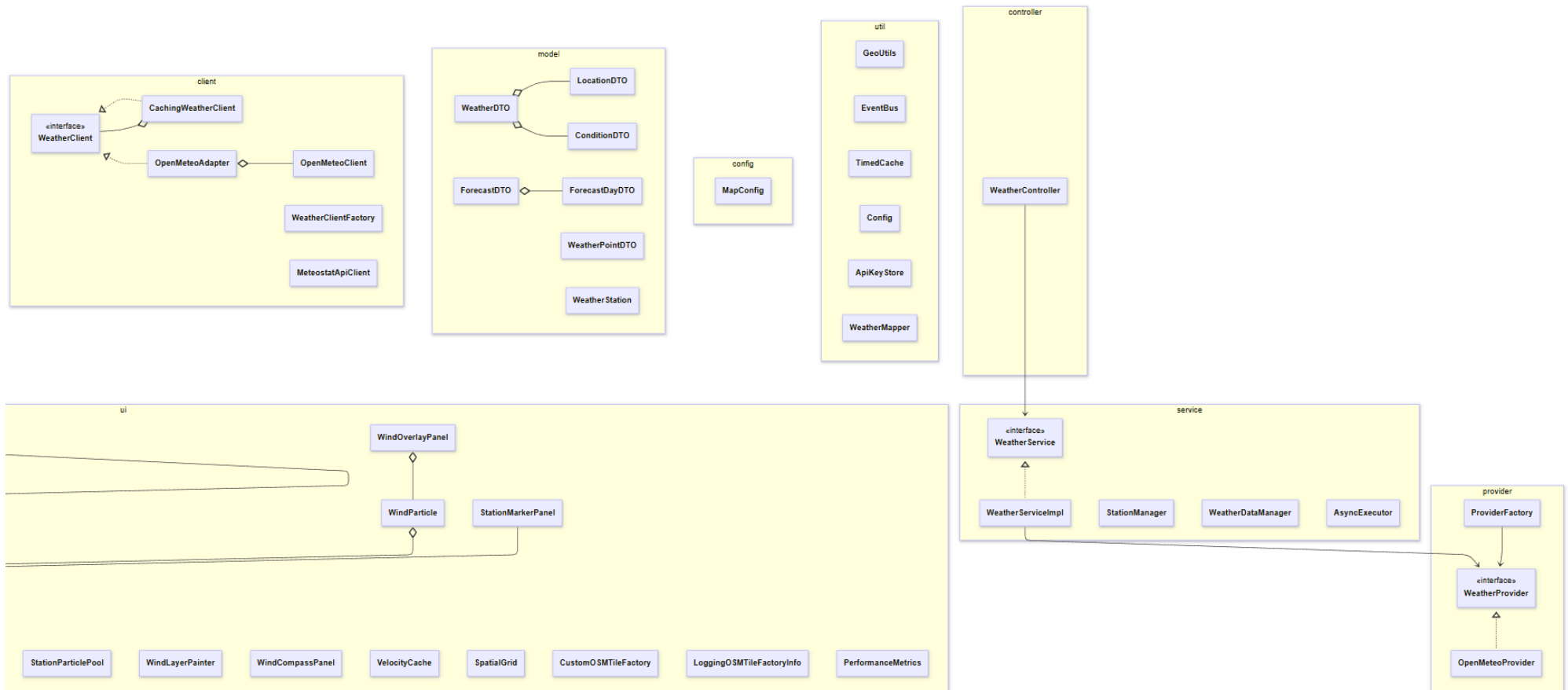
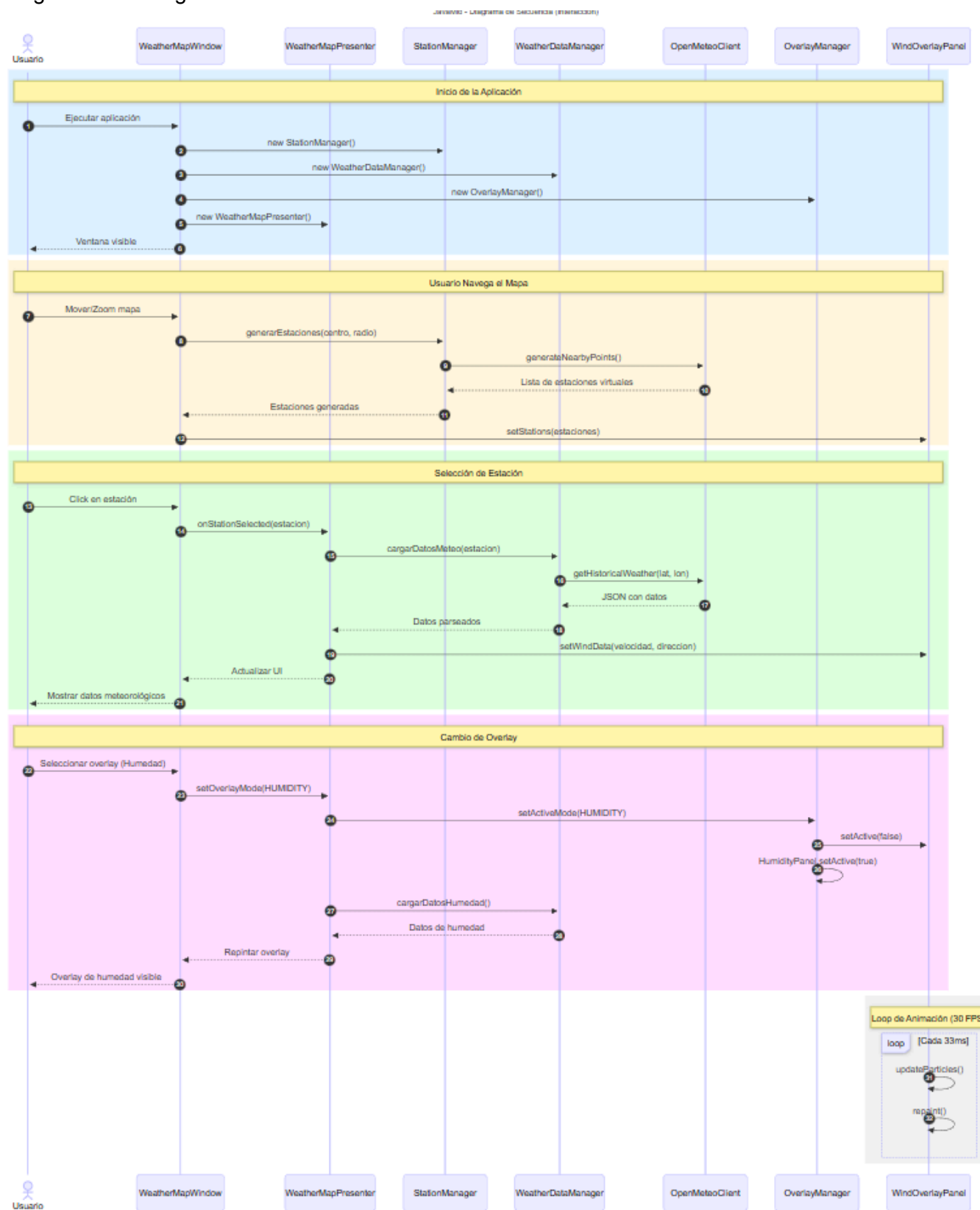


Diagrama 5.3. Diagrama de clases de la aplicacion (detalle derecha)

• **Diagramas de Comportamiento:**
Diagrama 5.4. Diagrama de Secuencia.



6) Repositorio git del proyecto:

<https://github.com/HectorLMF/Meteorologic-Map>